# Inertial Filter

*Component Design Document*

## 1 Description

Inertial attitude filter. Estimates the body attitude and body rate from the star tracker attitude and rate with a square root unscented Kalman filter, and publishes the estimate for the guidance and control algorithms. The filter state, the variance of each state, and the residuals of each measurement are published as data products. The filter synchronizes its time to the first tick after it is built or reset and propagates from there, so the star tracker time tag and the tick's call time must be on the same clock. Wraps the InertialFilterAlgorithm C++ algorithm via its C shim.

## 2 Requirements

No requirements have been specified for this component.

## 3 Design

### 3.1 At a Glance

Below is a list of useful parameters and statistics that give a quick look into the makeup of the component.

- **Execution** - *passive*
- **Number of Connectors** - 6
- **Number of Invokee Connectors** - 4
- **Number of Invoker Connectors** - 2
- **Number of Generic Connectors** - *None*
- **Number of Generic Types** - *None*
- **Number of Unconstrained Arrayed Connectors** - *None*
- **Number of Commands** - *None*
- **Number of Parameters** - 7
- **Number of Events** - *None*
- **Number of Faults** - *None*
- **Number of Data Products** - 4
- **Number of Data Dependencies** - 1
- **Number of Packets** - *None*

## 3.2 Diagram

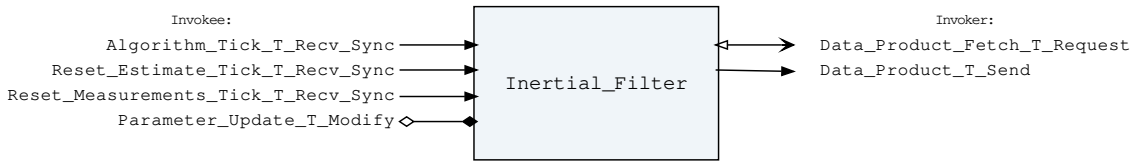



Figure 1: Inertial Filter component diagram.

## 3.3 Connectors

Below are tables listing the component's connectors.

### 3.3.1 Invokee Connectors

The following is a list of the component's *invokee* connectors:

Table 1: Inertial Filter Invokee Connectors

| Name | Kind | Type | Return_Type | Count |
|---|---|---|---|---|
| Algorithm_Tick_T_Recv_Sync | recv_sync | Algorithm_Tick.T | - | 1 |
| Reset_Estimate_Tick_T_Recv_Sync | recv_sync | Tick.T | - | 1 |
| Reset_Measurements_Tick_T_Recv_Sync | recv_sync | Tick.T | - | 1 |
| Parameter_Update_T_Modify | modify | Parameter_Update.T | - | 1 |

Connector Descriptions:

- **Algorithm_Tick_T_Recv_Sync** - Run the filter up to the tick's call time, folding in a fresh star tracker reading when there is one.
- **Reset_Estimate_Tick_T_Recv_Sync** - Re-seed the filter state and covariance from the configured initial values and clear the pending measurements and residuals. The filter synchronizes its time again at the next tick. The assembly fires this on every entry to the navigation state, where the filter is first put into use.
- **Reset_Measurements_Tick_T_Recv_Sync** - Clear the pending measurements and residuals, keeping the filter state and covariance. The filter synchronizes its time again at the next tick. The assembly fires this on every transition between pointing states.
- **Parameter_Update_T_Modify** - The parameter update connector.

### 3.3.2 Invoker Connectors

The following is a list of the component's *invoker* connectors:

Table 2: Inertial Filter Invoker Connectors

| Name | Kind | Type | Return_Type | Count |
|---|---|---|---|---|
| Data_Product_Fetch_T_Request | request | Data_Product_Fetch.T | Data_Product_Return.T | 1 |

| | | | | |
|---|---|---|---|---|
| Data_Product_T_Send | send | Data_Product.T | - | 1 |

Connector Descriptions:

- **Data_Product_Fetch_T_Request** - Fetch a data product item from the database.
- **Data_Product_T_Send** - The data product invoker connector

## 3.4 Interrupts

This component contains no interrupts.

## 3.5 Initialization

Below are details on how the component should be initialized in an assembly.

### 3.5.1 Component Instantiation

This component contains no instantiation parameters in its discriminant.

### 3.5.2 Component Base Initialization

This component contains no base class initialization, meaning there is no `init_Base` subprogram for this component.

### 3.5.3 Component Set ID Bases

This component contains commands, events, packets, faults, or data products that require a base identifier to be set at initialization. The `set_Id_Bases` procedure must be called with the following parameters:

Table 3: Inertial Filter Set Id Bases Parameters

| Name | Type |
|---|---|
| Data_Product_Id_Base | Data_Product_Types.Data_Product_Id_Base |
| Parameter_Id_Base | Parameter_Types.Parameter_Id_Base |

Parameter Descriptions:

- **Data_Product_Id_Base** - The value at which the component's data product identifiers begin.
- **Parameter_Id_Base** - The value at which the component's parameter identifiers begin.

### 3.5.4 Component Map Data Dependencies

This component contains data dependencies that need their IDs mapped to valid data product IDs at initialization. Each data dependency also needs a stale limit provided in microseconds. If the timestamp of the data dependency is older than the reference time (usually the current time) minus the stale limit then the data dependency is considered too old to safely use. The `map_Data_Dependencies` procedure must be called with the following parameters:

Table 4: Inertial Filter Map Data Dependencies Parameters

| Name | Type |
|---|---|
| Star_Tracker_Attitude_Id | Data_Product_Types.Data_Product_Id |

| Star_Tracker_Attitude_Stale_Limit | Ada.Real_Time.Time_Span |
|---|---|

Parameter Descriptions:

- **Star_Tracker_Attitude_Id** - The external data product ID used to fetch the value for the 'Star_Tracker_Attitude' data dependency.
- **Star_Tracker_Attitude_Stale_Limit** - The stale limit used for the 'Star_Tracker_Attitude' data dependency. If the fetched data product is older than the given Time_Span then it is deemed stale and unfit for use by the component.

#### 3.5.5 Component Implementation Initialization

The calling of this implementation class initialization procedure is mandatory. Initializes the inertial filter with the default parameter values, which seed the filter state and covariance. The `init` subprogram takes no parameters.

## 3.6 Commands

The Inertial Filter component has no commands.

## 3.7 Parameters

Parameters for the Inertial Filter component. The process noise and the initial covariance are given by their diagonals, since the full matrices do not fit in a parameter.

Table 5: Inertial Filter Parameters

| Local ID | Parameter Name | Type | Default Value |
|---|---|---|---|
| 0x0000 (0) | Alpha | Packed_F64.T | (Value=>0.02) |
| 0x0001 (1) | Beta | Packed_F64.T | (Value=>2.0) |
| 0x0002 (2) | Process_Noise_Diagonal | Packed_F64x6.T | [1.0E-14, 1.0E-14, 1.0E-14, 1.0E-16, 1.0E-16,1.0E-16] |
| 0x0003 (3) | Initial_State | Packed_F64x6.T | [0.0,0.0,0.0, 0.0,0.0,0.0] |
| 0x0004 (4) | Initial_Covariance_Diagonal | Packed_F64x6.T | [1.0E-4,1.0E-4,1.0E-4, 1.0E-4,1.0E-4,1.0E-4] |
| 0x0005 (5) | St_Measurement_Noise_Std | Packed_F64.T | (Value=>1.0E-4) |
| 0x0006 (6) | Rate_Measurement_Noise_Std | Packed_F64.T | (Value=>0.001) |

Parameter Descriptions:

- **Alpha** - [-] Sigma point spread. Must be in (0, 1].
- **Beta** - [-] Prior knowledge tunable. Must be in [0, 2].
- **Process_Noise_Diagonal** - [-] Diagonal of the N x N process noise covariance, one entry per state: the attitude MRP followed by the body rate. Every entry must be finite and not negative.

- **`Initial_State`** - [-] State the filter is seeded with, the attitude MRP followed by the body rate in rad/s. Must be finite.
- **`Initial_Covariance_Diagonal`** - [-] Diagonal of the N x N covariance the filter is seeded with, one entry per state. Every entry must be finite and not negative.
- **`St_Measurement_Noise_Std`** - [-] Standard deviation of the star tracker attitude measurement noise. Must not be negative.
- **`Rate_Measurement_Noise_Std`** - [rad/s] Standard deviation of the rate measurement noise. Must not be negative.

## 3.8 Events

The Inertial Filter component has no events.

## 3.9 Data Products

Data products for the Inertial Filter component.

Table 6: Inertial Filter Data Products

| Local ID | Data Product Name | Type |
|---|---|---|
| 0x0000 (0) | Attitude_Estimate | Nav_Att_Output.T |
| 0x0001 (1) | Filter_State | Inertial_Filter_State.T |
| 0x0002 (2) | St_Att_Residuals | Inertial_Filter_Fit_Residuals.T |
| 0x0003 (3) | Rate_Residuals | Inertial_Filter_Fit_Residuals.T |

Data Product Descriptions:

- **`Attitude_Estimate`** - Estimated body attitude and body rate, stamped with the time the filter advanced to. The sun direction is not estimated by this filter and is zero.
- **`Filter_State`** - The filter state after the update, the variance of each state, and the system time of the last star tracker reading applied.
- **`St_Att_Residuals`** - Star tracker attitude residuals of this update, with whether that measurement fired.
- **`Rate_Residuals`** - Rate residuals of this update, with whether that measurement fired.

## 3.10 Data Dependencies

Data dependencies for the Inertial Filter component.

Table 7: Inertial Filter Data Dependencies

| Data Dependency Name | Type |
|---|---|
| Star_Tracker_Attitude | St_Att.T |

Data Dependency Descriptions:

- **`Star_Tracker_Attitude`** - Star tracker attitude and body rate in the body frame, with the time tag of the measurement on the system clock. Both are fed to the filter as measurements when the time tag has advanced since the last reading consumed.

### 3.11 Packets

The Inertial Filter component has no packets.

### 3.12 Faults

The Inertial Filter component has no faults.

## 4 Unit Tests

The following section describes the unit test suites written to test the component.

### 4.1 *Inertial_ Filter_ Tests* Test Suite

This is a unit test suite for the Inertial Filter component

Test Descriptions:

- **`Test`** - Feed a constant star tracker attitude and check the estimate converges to it while the covariance shrinks, as in the algorithm's Python reference test, to ensure the Ada to C to C++ integration is sound. The filter state and residual products report the update and its time.
- **`Test_Rate_Update`** - Feed a constant body rate about one axis with the attitude that rotation gives, and check the rate estimate converges to it while the attitude tracks the truth.
- **`Test_Propagation`** - Ensure a star tracker product whose time tag has not advanced is not fed to the filter, so the estimate holds and the covariance carries the process noise.
- **`Test_Reset`** - Ensure the estimate reset re-seeds the state and covariance while the measurements reset keeps them.
- **`Test_Reading_Before_Sync`** - Ensure a star tracker reading not refreshed since before a reset is not consumed again, one stamped before the tick the filter synchronized its time to is consumed and dropped, and a later one is applied.
- **`Test_Time_Anomalies`** - Ensure a reading stamped after the tick is applied at once without shutting out the readings that follow for good, and a product stale between star tracker solutions is still read.
- **`Test_Parameter_Update`** - Ensure a parameter update reaches the filter while keeping the current estimate, with the new seed showing only at the next estimate reset.
- **`Test_Invalid_Parameter`** - Ensure a staged configuration the algorithm would reject is refused at validation.
- **`Test_Invalid_Data_Dependency`** - Ensure a data dependency with the wrong identifier is treated as a wiring defect and fails the tick's assertion.

## 5 Appendix

### 5.1 Preamble

This component contains no preamble code.

### 5.2 Packed Types

The following section outlines any complex data types used in the component in alphabetical order. This includes packed records and packed arrays that might be used as connector types, command arguments, event parameters, etc..

## Algorithm_Tick.T:

The tick an algorithm component runs on, with the time the algorithm propagates to already converted to nanoseconds. The component that dispatches the algorithm ticks converts the tick time once and hands the same value to every algorithm it runs in that tick, so the clock the algorithms propagate on is chosen in exactly one place.

Table 8: Algorithm_Tick Packed Record : 144 bits

| Name | Type | Range | Size (Bits) | Start Bit | End Bit |
|---|---|---|---|---|---|
| Current_Tick | Tick.T | - | 80 | 0 | 79 |
| Call_Time | Interfaces. Unsigned_64 | 0 to 18446744073709551615 | 64 | 80 | 143 |

Field Descriptions:

- **Current_Tick** - The tick as scheduled. Algorithm components stamp their data products with its time and use it as the stale reference for their data dependencies.
- **Call_Time** - [ns] The time the algorithm propagates to, in nanoseconds of the tick time.

## Data_Product.T:

Generic data product packet for holding arbitrary data types

Table 9: Data_Product Packed Record : 1096 bits *(maximum)*

| Name | Type | Range | Size (Bits) | Start Bit | End Bit | Variable Length |
|---|---|---|---|---|---|---|
| Header | Data_Product_ Header.T | - | 72 | 0 | 71 | - |
| Buffer | Data_Product_ Types.Data_ Product_ Buffer_Type | - | 1024 | 72 | 1095 | Header. Buffer_Length |

Field Descriptions:

- **Header** - The data product header
- **Buffer** - A buffer that contains the data product type

## Data_Product_Fetch.T:

A packed record which holds information for a data product request.

Table 10: Data_Product_Fetch Packed Record : 16 bits

| Name | Type | Range | Size (Bits) | Start Bit | End Bit |
|---|---|---|---|---|---|
| Id | Data_Product_Types. Data_Product_Id | 0 to 65535 | 16 | 0 | 15 |

Field Descriptions:

- **Id** - The data product identifier

## Data_Product_Header.T:

Generic data_product packet for holding arbitrary data_product types

Table 11: Data_Product_Header Packed Record : 72 bits

| Name | Type | Range | Size (Bits) | Start Bit | End Bit |
|---|---|---|---|---|---|
| Time | Sys_Time.T | - | 48 | 0 | 47 |
| Id | Data_Product_Types.<br>Data_Product_Id | 0 to 65535 | 16 | 48 | 63 |
| Buffer_Length | Data_Product_<br>Types.Data_Product_<br>Buffer_Length_Type | 0 to 128 | 8 | 64 | 71 |

Field Descriptions:

- **Time** - The timestamp for the data product item.
- **Id** - The data product identifier
- **Buffer_Length** - The number of bytes used in the data product buffer

## Data_Product_Return.T:

This record holds data returned from a data product fetch request.

Table 12: Data_Product_Return Packed Record : 1104 bits *(maximum)*

| Name | Type | Range | Size (Bits) | Start Bit | End Bit | Variable Length |
|---|---|---|---|---|---|---|
| The_<br>Status | Data_<br>Product_<br>Enums.<br>Fetch_<br>Status.E | 0 => Success<br>1 => Not_Available<br>2 => Id_Out_Of_Range | 8 | 0 | 7 | - |
| The_<br>Data_<br>Product | Data_<br>Product.<br>T | - | 1096 | 8 | 1103 | - |

Field Descriptions:

- **The_Status** - A status relating whether or not the data product fetch was successful or not.
- **The_Data_Product** - The data product item returned.

## Inertial_Filter_Fit_Residuals.T:

Residuals of one measurement kind from one inertial filter update, before and after the update. Built from the residuals the inertialFilter C shim returns, without the measured value, which is already published by the sensor product.

Table 13: Inertial_Filter_Fit_Residuals Packed Record : 392 bits

| Name | Type | Range | Size (Bits) | Start Bit | End Bit |
|---|---|---|---|---|---|
| Valid | Boolean | 0 => False<br>1 => True | 8 | 0 | 7 |
| Pre_Fit | Packed_F64x3.T | - | 192 | 8 | 199 |
| Post_Fit | Packed_F64x3.T | - | 192 | 200 | 391 |

Field Descriptions:

- **Valid** - Whether this measurement fired and was applied this update. The residuals are meaningful only when it did.
- **Pre_Fit** - [-] Residual before the update.
- **Post_Fit** - [-] Residual after the update.

## Inertial_Filter_State.T:

State of the inertial filter after an update, the estimated state, the variance of each state, and when the last star tracker reading was consumed. Built from the snapshot the inertialFilter C shim returns, with the N x N covariance reduced to its diagonal.

Table 14: Inertial_Filter_State Packed Record : 832 bits

| Name | Type | Range | Size (Bits) | Start Bit | End Bit |
|---|---|---|---|---|---|
| State | Packed_F64x6.T | - | 384 | 0 | 383 |
| Covariance_Diagonal | Packed_F64x6.T | - | 384 | 384 | 767 |
| Last_Measurement_Time | Interfaces.Unsigned_64 | 0 to 18446744073709551615 | 64 | 768 | 831 |

Field Descriptions:

- **State** - [-] Estimated state, the body attitude MRP followed by the body rate in rad/s.
- **Covariance_Diagonal** - [-] Diagonal of the state covariance, the variance of each state.
- **Last_Measurement_Time** - [ns] System time of the last star tracker reading consumed by the filter, zero until one arrives after the filter is built or reset. Tells when the star tracker last fed the filter once this product is sampled into a packet.

## Nav_Att_Output.T:

Table 15: Nav_Att_Output Packed Record : 352 bits

| Name | Type | Range | Size (Bits) | Start Bit | End Bit |
|---|---|---|---|---|---|
| Time_Tag | Long_Float | -inf to inf | 64 | 0 | 63 |
| Sigma_Bn | Packed_F32x3.T | - | 96 | 64 | 159 |
| Omega_Bn_B | Packed_F32x3.T | - | 96 | 160 | 255 |
| Veh_Sun_Pnt_Bdy | Packed_F32x3.T | - | 96 | 256 | 351 |

Field Descriptions:

- **Time_Tag** - [s] Time tag of navigation solution
- **Sigma_Bn** - MRP attitude of body w.r.t. inertial frame
- **Omega_Bn_B** - [rad/s] Angular velocity of body w.r.t. inertial, body frame components
- **Veh_Sun_Pnt_Bdy** - Sun direction unit vector in body frame

## Packed_F32x3.T:

Packed 3 element vector of 32-bit floats.

Table 16: Packed_F32x3 Packed Array : 96 bits

| Type | Range | Element Size (Bits) | Length | Total Size (Bits) |
|---|---|---|---|---|
| **Short_Float** | -3.40282e+38 to 3.40282e+38 | 32 | 3 | 96 |

## Packed_F64.T:

Single component record for holding packed 64-bit floating point number.

Table 17: Packed_F64 Packed Record : 64 bits

| Name | Type | Range | Size (Bits) | Start Bit | End Bit |
|---|---|---|---|---|---|
| Value | Long_Float | -inf to inf | 64 | 0 | 63 |

Field Descriptions:

- **Value** - The 64-bit floating point number.

## Packed_F64x3.T:

Packed 3 element vector of 64-bit floats.

Table 18: Packed_F64x3 Packed Array : 192 bits

| Type | Range | Element Size (Bits) | Length | Total Size (Bits) |
|---|---|---|---|---|
| **Long_Float** | -inf to inf | 64 | 3 | 192 |

## Packed_F64x6.T:

Packed 6 element vector of 64-bit floats.

Table 19: Packed_F64x6 Packed Array : 384 bits

| Type | Range | Element Size (Bits) | Length | Total Size (Bits) |
|---|---|---|---|---|
| **Long_Float** | -inf to inf | 64 | 6 | 384 |

## Parameter.T:

Generic parameter packet for holding a generic parameter

Table 20: Parameter Packed Record : 792 bits *(maximum)*

| Name | Type | Range | Size (Bits) | Start Bit | End Bit | Variable Length |
|---|---|---|---|---|---|---|
| Header | Parameter_ Header.T | - | 24 | 0 | 23 | - |
| Buffer | Parameter_ Types. Parameter_ Buffer_Type | - | 768 | 24 | 791 | Header.Buffer_ Length |

Field Descriptions:

- **`Header`** - The parameter header
- **`Buffer`** - A buffer that contains the parameter type

## Parameter_Header.T:

Generic parameter header for holding arbitrary parameters

Table 21: Parameter_Header Packed Record : 24 bits

| Name | Type | Range | Size (Bits) | Start Bit | End Bit |
|---|---|---|---|---|---|
| Id | Parameter_Types.Parameter_Id | 0 to 65535 | 16 | 0 | 15 |
| Buffer_Length | Parameter_Types.Parameter_Buffer_Length_Type | 0 to 96 | 8 | 16 | 23 |

Field Descriptions:

- **`Id`** - The parameter identifier
- **`Buffer_Length`** - The number of bytes used in the parameter type buffer

## Parameter_Update.T:

A record intended to be used as a provide/modify connector type for updating/fetching parameters.

Table 22: Parameter_Update Packed Record : 824 bits *(maximum)*

| Name | Type | Range | Size (Bits) | Start Bit | End Bit | Variable Length |
|---|---|---|---|---|---|---|
| Table_Id | Parameter_Types.Parameter_Table_Id | 0 to 65535 | 16 | 0 | 15 | - |
| Operation | Parameter_Enums.Parameter_Operation_Type.E | 0 => Stage<br>1 => Update<br>2 => Fetch<br>3 => Validate | 8 | 16 | 23 | - |
| Status | Parameter_Enums.Parameter_Update_Status.E | 0 => Success<br>1 => Id_Error<br>2 => Validation_Error<br>3 => Length_Error | 8 | 24 | 31 | - |
| Param | Parameter.T | - | 792 | 32 | 823 | - |

Field Descriptions:

- **Table_Id** - The ID for the table that contains this parameter
- **Operation** - The parameter operation to perform.
- **Status** - The parameter return status.
- **Param** - The parameter that has been updated or fetched.

## St_Att.T:

Table 23: St_Att Packed Record : 256 bits

| Name | Type | Range | Size (Bits) | Start Bit | End Bit |
|---|---|---|---|---|---|
| Time_Tag | Interfaces. Unsigned_64 | 0 to 18446744073709551615 | 64 | 0 | 63 |
| Sigma_Bn | Packed_ F32x3.T | - | 96 | 64 | 159 |
| Omega_Bn_B | Packed_ F32x3.T | - | 96 | 160 | 255 |

Field Descriptions:
- **Time_Tag** - [ns] Time tag associated with measurement
- **Sigma_Bn** - [-] MRP from inertial to body frame
- **Omega_Bn_B** - [rad/s] Body-frame angular velocity w.r.t. inertial

## Sys_Time.T:

A record which holds a time stamp using EMA VTC format including seconds and subseconds since epoch (1-5-1980 to 1-6-1980 midnight).

Table 24: Sys_Time Packed Record : 48 bits

| Name | Type | Range | Size (Bits) | Start Bit | End Bit |
|---|---|---|---|---|---|
| Seconds | Interfaces. Unsigned_32 | 0 to 4294967295 | 32 | 0 | 31 |
| Subseconds | Interfaces. Unsigned_16 | 0 to 65535 | 16 | 32 | 47 |

Field Descriptions:
- **Seconds** - The number of seconds elapsed since epoch.
- **Subseconds** - The number of 1/(2^16) sub-seconds.

## Tick.T:

The tick datatype used for periodic scheduling. Included in this type is the Time associated with a tick and a count.

Table 25: Tick Packed Record : 80 bits

| Name | Type | Range | Size (Bits) | Start Bit | End Bit |
|---|---|---|---|---|---|
| Time | Sys_Time.T | - | 48 | 0 | 47 |

| | | | | | |
|---|---|---|---|---|---|
| Count | Interfaces.<br>Unsigned_32 | 0 to 4294967295 | 32 | 48 | 79 |

Field Descriptions:

- **Time** - The timestamp associated with the tick.
- **Count** - The cycle number of the tick.

## 5.3 Enumerations

The following section outlines any enumerations used in the component.

### Data_Product_Enums.Fetch_Status.E:

This status denotes whether a data product fetch was successful.

Table 26: Fetch_Status Literals:

| Name | Value | Description |
|---|---|---|
| Success | 0 | The data product was returned successfully. |
| Not_Available | 1 | No data product is yet available for the provided id. |
| Id_Out_Of_Range | 2 | The data product id was out of range. |

### Parameter_Enums.Parameter_Operation_Type.E:

This enumeration lists the different parameter operations that can be performed.

Table 27: Parameter_Operation_Type Literals:

| Name | Value | Description |
|---|---|---|
| Stage | 0 | Stage the parameter. |
| Update | 1 | All parameters are staged, it is ok to update all parameters now. |
| Fetch | 2 | Fetch the parameter. |
| Validate | 3 | Validate all the parameters. |

### Parameter_Enums.Parameter_Update_Status.E:

This status enumeration provides information on the success/failure of a parameter operation.

Table 28: Parameter_Update_Status Literals:

| Name | Value | Description |
|---|---|---|
| Success | 0 | Parameter was successfully staged. |
| Id_Error | 1 | Parameter id was not valid. |
| Validation_Error | 2 | Parameter values were not successfully validated. |
| Length_Error | 3 | Parameter length was not correct. |